

TASK 01: BURP SUITE MASTERY – THE INTERCEPTOR

The first task focused on configuring and mastering Burp Suite Professional as a web application proxy. Burp Suite is an industry-standard tool used by professional penetration testers to intercept, inspect, modify, and replay HTTP/HTTPS traffic between the browser and the target server. Understanding how to manipulate the Request-Response cycle is the foundation of all web application testing.

Phase 1: Request Interception via Burp Proxy

I configured Burp Suite's built-in browser and set the proxy listener to port 8080. By enabling 'Intercept is ON' in the Proxy tab, I was able to capture every HTTP request sent by the browser before it reached the server. The screenshot below shows a live intercepted GET request to rubinoshoes.com, where I can see the full request headers, query parameters, and session cookies in real time. The Inspector panel on the right breaks down the request into structured components such as query parameters, cookies, and headers — making it easy to identify injection points.

The screenshot displays the Burp Suite Proxy Intercept tab. At the top, there's a toolbar with 'Intercept on' (checked), 'Forward', and 'Drop' buttons. Below this is a table of intercepted requests:

Time	Type	Direction	Method	URL	Status code	Length
13:08:19 1 Apr 2026	HTTP	→ Request	GET	https://rubinoshoes.com/?srsltid=AfmBOooxVk4cVElGNMG80f_4kmSVRVvdv01XIAnCTVkt8b3rwwyYDle		
13:08:22 1 Apr 2026	HTTP	→ Request	GET	https://push.services.mozilla.com/		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://location.services.mozilla.com/v1/country?key=91e66841-a83b-487f-9b5d-e460f5225ebf		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://services.addons.mozilla.org/api/v4/addons/search/?guid=default-theme%40mozilla.org%2Cfirefox-com...		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://firefox.settings.services.mozilla.com/v1/buckets/monitor/collections/changes/changeset?_expected=0		
13:08:36 1 Apr 2026	HTTP	→ Request	GET	https://normandy.cdn.mozilla.net/api/v1/		

The selected request is expanded in the 'Request' panel, showing the raw HTTP text:

```

1 GET /?srsltid=AfmBOooxVk4cVElGNMG80f_4kmSVRVvdv01XIAnCTVkt8b3rwwyYDle HTTP/1.1
2 Host: rubinoshoes.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Upgrade-Insecure-Requests: 1
8 Sec-Fetch-Dest: document
9 Sec-Fetch-Mode: navigate
10 Sec-Fetch-Site: none
11 Sec-Fetch-User: ?1
12 Priority: u=0, i
13 Te: trailers
14 Connection: keep-alive
15
16

```

The 'Inspector' panel on the right provides a structured view of the request components:

- Request attributes: 2
- Request query parameters: 1
- Request body parameters: 0
- Request cookies: 0
- Request headers: 13

Figure 1 – Burp Suite Proxy Intercept tab showing a captured HTTP GET request to rubinoshoes.com with full header and cookie visibility. The Inspector panel reveals 2 request attributes, 1 query parameter, 0 body parameters, and 13 request headers.

Phase 2: Request Modification & Response Analysis in Repeater

After intercepting a POST request, I forwarded it to the Burp Repeater module. Repeater allows me to manually modify any part of the request and re-send it as many times as needed, observing how the server responds to different inputs. This is critical for manual vulnerability testing. I selected specific form-data fields highlighted in the request body — particularly the 'quantity', 'product-id', and 'section-id' parameters — which are common injection points for business logic flaws and parameter tampering.

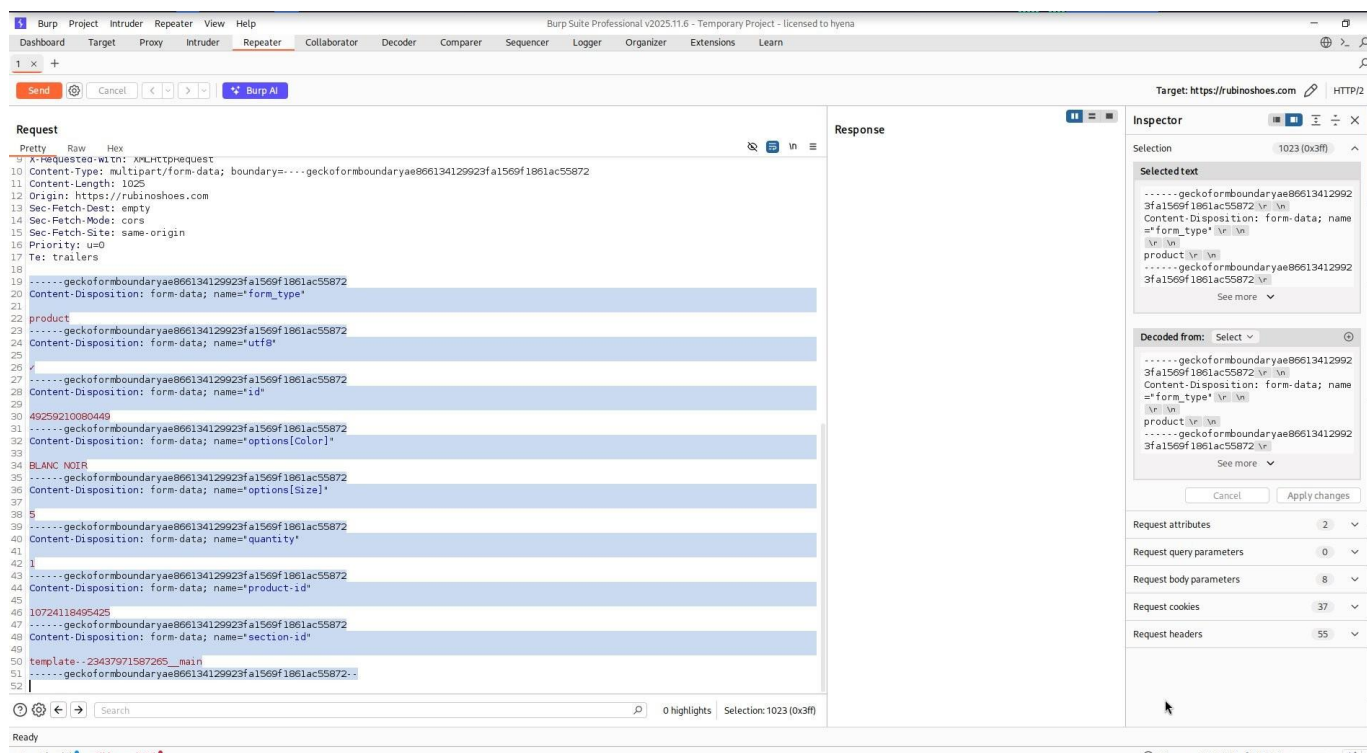


Figure 2 – Burp Suite Repeater showing a POST multipart/form-data request with the full body selected. The Inspector panel confirms 8 body parameters including id, options[Color], options[Size], quantity, product-id, and section-id — all potential injection or manipulation points.

Phase 3: Response Analysis After Modification

After sending the modified request, I analyzed the server's HTTP response in the Repeater panel. The server returned HTTP/2 200 OK with a Set-Cookie header containing a new Shopify session token and cart data. This confirmed that the server accepted the modified request. The response headers also revealed valuable information about the backend stack: Powered-By: Shopify, Cloudflare CDN, and X-Frame-Options: DENY. Observing server responses to modified requests is a core technique for mapping application behavior and identifying logic flaws.

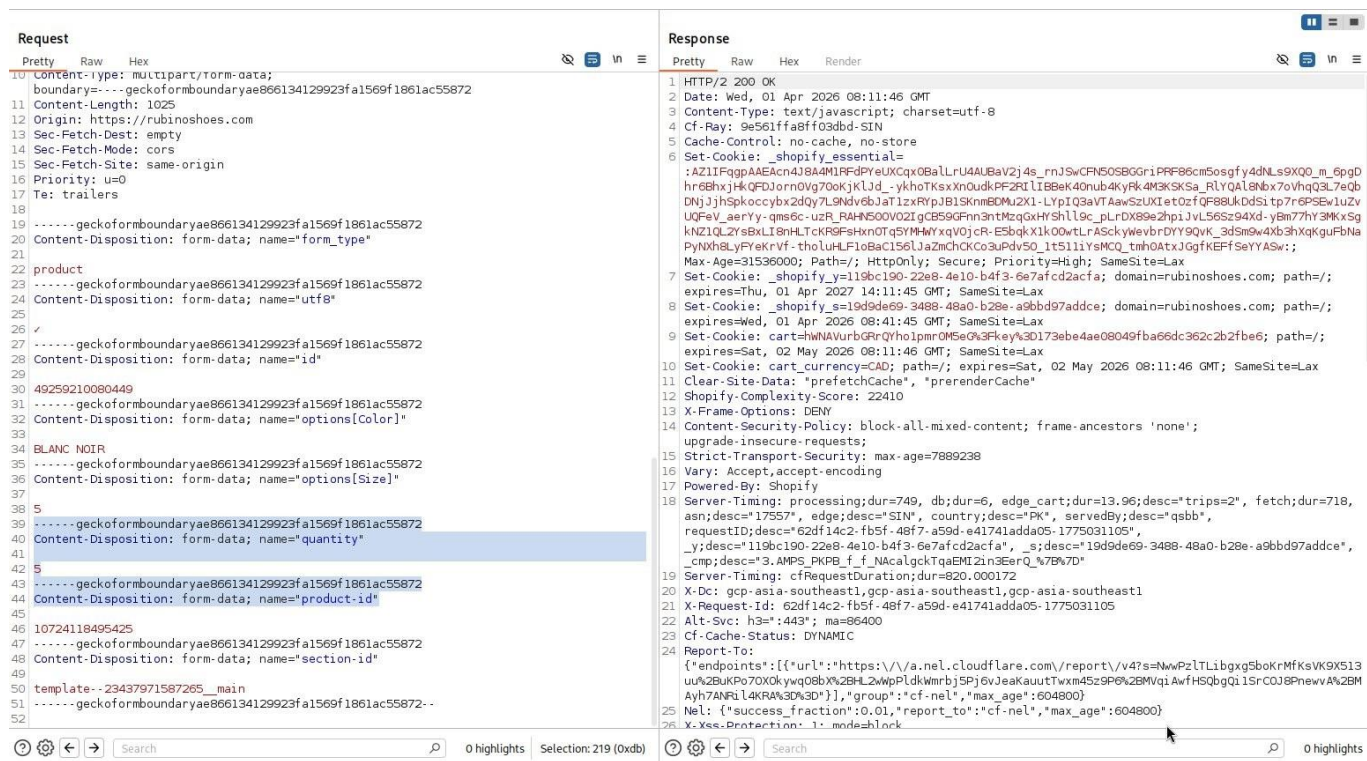


Figure 3 – Burp Suite Repeater Response panel showing HTTP/2 200 OK returned after the modified POST request. Multiple Set-Cookie headers reveal session tokens and cart data, and the Powered-By header confirms Shopify as the backend platform.

Phase 4: Automated Parameter Scanning with Burp Intruder

I then used Burp Intruder to perform automated parameter-based scanning. I configured a Sniper attack type with a numeric payload ranging from 1 to 10, targeting the 'id' parameter in the intercepted request body. Intruder automatically sent 10 requests, each with a different payload value. This technique is used to enumerate valid product IDs, user accounts, or other numeric identifiers — a common first step in discovering IDOR (Insecure Direct Object Reference) vulnerabilities. The attack window shows all 10 request-response pairs with their respective status codes.

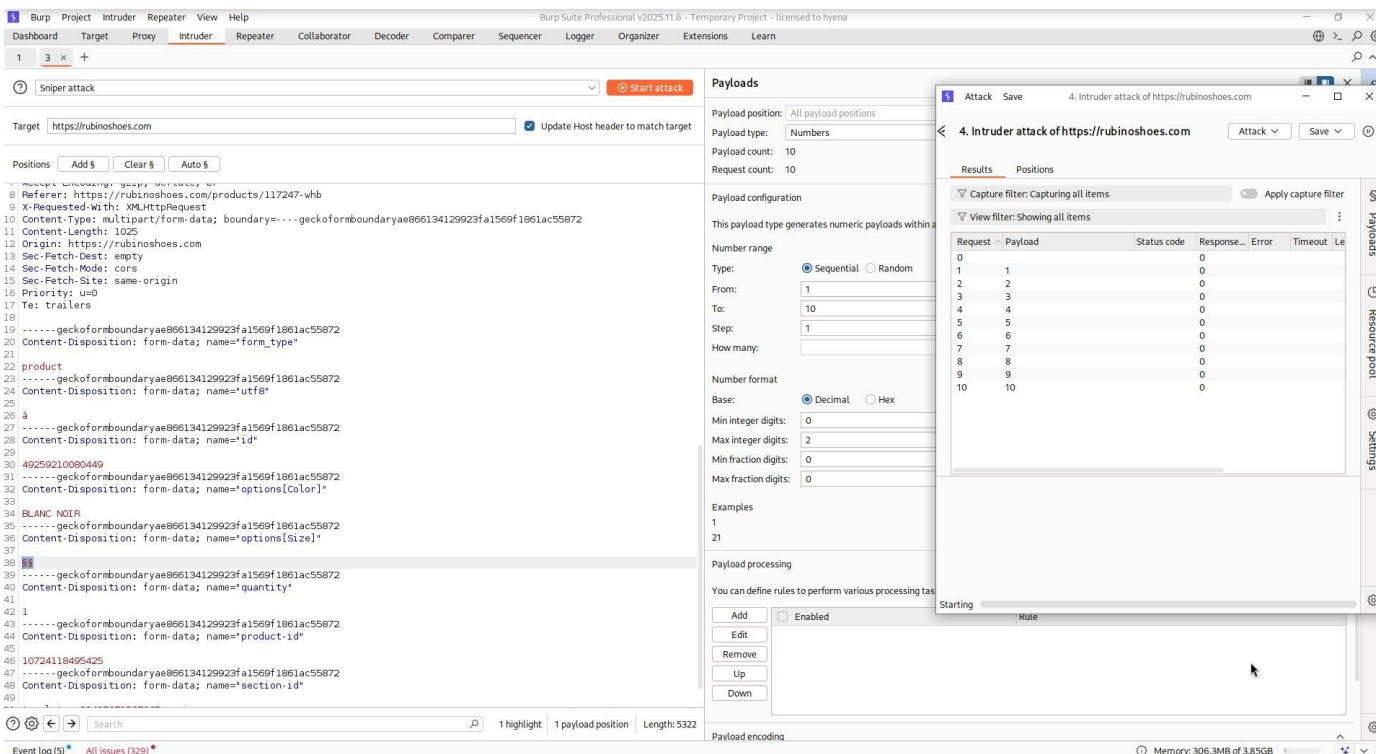


Figure 4 – Burp Suite Intruder configured with Sniper attack type, numeric payload (1-10, sequential, decimal), targeting the id parameter in a POST request to rubinoshoes.com. The attack results panel shows all 10 payload iterations.

Key Takeaway

Burp Suite is the command-and-control center of web application pentesting. The Proxy captures traffic invisibly, Repeater enables precise manual testing, and Intruder automates repetitive fuzzing tasks. Together they form the complete workflow for identifying and exploiting web vulnerabilities, which is exactly what I applied in the remaining three tasks this week.